

The Point of Abstractions

Or: How to Make Things Easy to Change

Agenda

- **Why abstractions?** ETC – Easy To Change
- **The domain model:** Types as documentation
- **Parse, don't validate:** Correctness at the boundary
- **Ports:** Contracts, not implementations
- **Use cases:** One job, no downstream dependencies
- **Adapters:** The things that do the actual work
- **The payoff:** From V1 to V2

Why abstractions?

Not for fun. Because things change

"ETC – Easy To Change. That's it. As far as we can tell, every design principle out there is a special case of ETC."

— The Pragmatic Programmer

-  Requirements change (always)
-  We need to test things in isolation
-  We need to swap out parts without tearing down the whole thing

The question isn't "do we need abstraction?"

The question is "does this abstraction make the code easier to change?"

The domain model: V1 vs V2

V1: Flat and implicit

- No `Conversation` model – just flat JSON blobs in the database
- Errors caught only at the route level
- `NetworkError(throwable)` as a catch-all

V2: Explicit and typed

```
data class Conversation(  
    val id: String,  
    val name: String,  
    val messages: List<MessageV2>,  
    val selectionID: String? = null  
)
```

The conversation is a first-class citizen in the domain.

Parse, don't validate

Enforce correctness **at the boundary** — then trust your types everywhere else.

```
data class CreateConversationRequest private constructor(  
    val query: String,  
    val selectionID: String? = null  
) {  
    companion object {  
        operator fun invoke(query: String, selectionID: String? = null) =  
            CreateConversationRequest(query = query.sanitized(), selectionID = selectionID)  
    }  
}  
  
sealed class MessageV2 {  
    data class User private constructor(val query: String) : MessageV2() {  
        companion object {  
            operator fun invoke(query: String) = User(query = query.sanitized())  
        }  
    }  
    // ...  
}
```

Private constructor + `operator fun invoke` = **impossible to create without parsing**

Parsing at the HTTP boundary

```
private data class CreateChatRequestDTO(val query: String, ...) {  
    fun toDomain(): Either<RouteError, CreateConversationRequest> =  
        either {  
            ensure(query.isNotEmpty()) { RouteError.BadRequest("...") }  
            ensure(query.toByteArray().size <= MAX_QUERY_SIZE) { RouteError.BadRequest("...") }  
            CreateConversationRequest(query = query, selectionID = selectionID)  
        }  
}
```

Raw input comes in as a DTO → validated and parsed into a domain type → or rejected.

Once you have a `CreateConversationRequest`, it's **guaranteed valid**.

No defensive checks deeper in the stack. No "just in case" validation.

Sealed classes: Making invalid states impossible

```
sealed class MessageV2 {  
    data class User(val query: String) : MessageV2()  
    data class System(  
        val response: XHTML.Node,  
        val relevantDocuments: List<RelevantDocument>  
    ) : MessageV2()  
    data class Error(val message: String) : MessageV2()  
}
```

A message is **either** from the user, **or** from the system, **or** an error. Never something in between.

- ✅ The compiler forces you to handle every variant
- ✅ New variants → compile error wherever you forgot

Type-driven state machine

```
private fun Conversation.toResponse() =  
    when (this.messages.lastOrNull()) {  
        null          -> ChatResponseDTO.InitialLoading(...)  
        is MessageV2.User    -> ChatResponseDTO.AwaitingReply(...)  
        is MessageV2.System -> ChatResponseDTO.Idle(...)  
        is MessageV2.Error   -> ChatResponseDTO.Failed(...)  
    }
```

No boolean flags. No `state: String`. Just types.

- ⚠️ V1: Frontend had to guess state based on JSON fields
- ✅ V2: State is **unambiguous** – it follows from the last message

Honest error handling

```
sealed interface AISearchErrorV2 {  
    data class Validation(val message: String) : AISearchErrorV2  
    data class NotFound(val message: String) : AISearchErrorV2  
    data object RateLimited : AISearchErrorV2  
    data class ServiceUnavailable(...) : AISearchErrorV2  
    data class Timeout(...) : AISearchErrorV2  
    data object TooLargeHeader : AISearchErrorV2  
    data class Internal(...) : AISearchErrorV2  
}
```

V1: `NetworkError(throwable)` – what actually went wrong?

V2: Every error type is explicit – the code *tells* you what can happen

 `Either<AISearchErrorV2, T>` — functions that don't lie

Ports

A port is a **contract** — an interface that defines *what* the system needs, not *how* it's done.

LLMPort

```
interface LLMPort {  
    suspend fun chat(  
        userID: Int,  
        conversation: Conversation  
    ): Either<AISearchErrorV2, Conversation>  
}
```

That's the whole thing.

- No HTTP clients, no JSON parsing, no retry logic
- Just: "give me a conversation back, or an error"

CachePort

```
interface CachePort {  
    suspend fun createConversation(  
        userID: Int, query: String, selectionID: String?  
    ): Either<AISearchErrorV2, Conversation>  
  
    suspend fun getConversation(  
        userID: Int, conversationID: String  
    ): Either<AISearchErrorV2, Conversation>  
  
    suspend fun updateConversation(  
        userID: Int, conversation: Conversation  
    ): Either<AISearchErrorV2, Unit>  
  
    suspend fun deleteConversation(  
        userID: Int, conversationID: String  
    ): Either<AISearchErrorV2, Unit>  
    // + list operations  
}
```

📦 CRUD contract for conversations — the implementation can be anything

Use cases

A use case does **one thing**.

It knows only ports — never concrete implementations.

PostMessage

```
class PostMessage(  
    private val llmPort: LLMPort,  
    private val cachePort: CachePort,  
    private val serverEventEmitter: ServerEventEmitter,  
    private val coroutinePool: CoroutinePool  
) {  
    suspend operator fun invoke(  
        userID: Int, sessionId: String,  
        conversationID: String, query: String  
    ): Either<AISearchErrorV2, Conversation> =  
        either {  
            val conversation =  
                cachePort.getConversation(userID, conversationID).bind()  
                    .dropErrors()  
                    .withMessage(MessageV2.User(query))  
                    .also { cachePort.updateConversation(userID, it).bind() }  
  
            coroutinePool.launch { /* LLM call in the background */ }  
            conversation  
        }  
}
```

What happens in PostMessage?

1. 📥 Fetch conversation from cache (CachePort)
2. 💬 Append the user's message
3. 💾 Save updated conversation (CachePort)
4. 🚀 Fire off LLM call in background (LLMPort + CoroutinePool)
5. 🔄 Return conversation immediately

The use case **orchestrates** — it doesn't know if cache is Postgres or a `HashMap`, or if the LLM is the Lovdata API or a mock.

Wiring

The routes wire everything together — the **only place** that knows which adapters are used.

Dependency injection in the routes

```
class AISearchRoutesV2 @Inject constructor(
    application: Application,
    llmPort: LLMPort,
    cachePort: CachePort,
    serverEventEmitter: ServerEventEmitter
) {
    private val coroutinePool = CoroutinePool(maxConcurrency = 50)

    private val createConversation = CreateConversation(llmPort, cachePort, ...)
    private val getConversation    = GetConversation(cachePort)
    private val postMessage       = PostMessage(llmPort, cachePort, ...)
    private val retryLastMessage  = RetryLastMessage(llmPort, cachePort, ...)
    private val deleteConversation = DeleteConversation(cachePort)
}
```

✿ Each use case gets the ports it needs — done.



Real adapter: LovdataApiAdapter

```
class LovdataApiAdapter @Inject constructor(  
    private val httpClient: HttpClient,  
    private val documentService: DocumentService  
) : LLMPort {  
  
    override suspend fun chat(userID: Int, conversation: Conversation) =  
        either {  
            val docs = findRelevantDocuments(userID, ...).bind()  
            val request = buildChatRequest(userID, conversation, docs)  
            val (response, name) = generateResponseAndName(request).bind()  
            conversation  
                .withNameIfNotNull(name)  
                .withMessage(MessageV2.System(response, docs))  
        }  
    // 300+ lines: HTTP calls, DTO mapping, error handling...  
}
```

🏢 All the complexity is **encapsulated** behind `LLMPort`

Mock adapter: `MockLLMAdapter`

```
class MockLLMAdapter : LLMPort {  
    override suspend fun chat(  
        userID: Int, conversation: Conversation  
    ): Either<AISearchErrorV2, Conversation> =  
        conversation.withMessage(mockResponse()).right()  
}
```

✅ Same interface — **5 lines** instead of 300+

The use cases can't tell the difference. That's the whole point.

What did we gain?

	V1	V2
Structure	One big AISearchService	Ports + Adapters + Usecases
Storage	Flat JSON blob	Normalized tables
Error handling	Route level only	In the domain model (Either)
Conversations	Don't exist	First-class aggregate
Swap LLM/search implementation	Rewrite the service	Write one new adapter
Testing	Mock the whole service	Inject a mock adapter

When requirements change...

"We're switching from lovdata-api to direct search or some other vastly different setup"

→ Write a new adapter that implements `LLMPort` . Done.

"We need a different storage solution"

→ Write a new adapter that implements `CachePort` . Done.

"We want to add a new feature"

→ Create a new use case. Existing code stays untouched.

No changes to domain logic. No changes to other adapters.

Testing without pain 🩺

Both implement the same interface:

```
interface LLMPort {  
    suspend fun chat(  
        userID: Int,  
        conversation: Conversation  
    ): Either<AISearchErrorV2, Conversation>  
}
```

4 lines that define the contract.

No `every { }`, `verify { }`, `@MockBean`.

Just a class that implements an interface.

ETC as a daily design habit 🚫

"ETC is a value, not a rule. [...] It's the ultimate arbiter between design alternatives."

— The Pragmatic Programmer

Not an architecture pattern you "follow" — a daily habit:

- **Before you write code:** *"Does this make it easier to change later?"*
- **When in doubt:** *"Which solution is easier to change?"*
- **When someone asks why:** *"Because change is cheap."*

Architecture isn't about being "clean" or "correct".

It's about making **the next change cheap**.

Thanks! 🙏

Questions?